Verifier-Guided Generate→Validate→Repair for Intent-Driven Network Configuration: A Confirmatory Study

Autonomous AI Researcher
CNSM 2026 Agentic AI Researcher Track

***Abstract*—We evaluate whether inserting a deterministic network verifier into an LLM-driven generate→validate→repair pipeline reduces functional regressions and blast radius versus LLM-alone proposals. Under a preregistered paired design (CNSM2026-A1-prereg-v1; preregistration SHA-256 feba320ea2e03679bd575ac3929f0dc060a5844982bec6758ac87d02a272eba3) we synthesized 200 stratified intent→configuration tasks and ran paired episodes with gpt-5-mini under two conditions: baseline (LLM-alone) and guarded (LLM + deterministic_netops_validator_v5 + up to one automated repair iteration). On 188 complete pairs (376 completed episodes; 24 failed episodes) guarded vs baseline success rates were 1.000 and 0.9840425531914894 respectively; paired difference = 0.015957446808510634, 95% bootstrap percentile CI [0.0, 0.03723404255319149], exact McNemar two-sided p = 0.25. The preregistered efficacy threshold (absolute change >= 0.20) was not met. Prespecified sensitivity (treating both-missing pairs as failures) yields guarded_success = 188/200 = 0.94, baseline_success = 185/200 = 0.925, paired difference Δ = 0.015; preregistered bootstrap percentile 95% CI (10,000 resamples, seed=1729) = [0.0, 0.035]. Repair activity was rare (repair invocations = 3) and corresponds to the three discordant guarded-only successes (n_10 = 3). We release the full artifact bundle for reproducibility and provide deterministic extraction and bootstrap recipes referencing archived artifact checksums in the Disclosure Statement.**

I. Introduction

Motivation and question. Large language models (LLMs) are increasingly proposed to translate operator intent into device configurations and to assist NetOps automation. Operational correctness requires preserving invariants, bounding blast radius, and producing auditable traces. A common architectural response is generate→validate→repair: an LLM proposes configuration(s), a deterministic verifier checks policy and functional invariants, and an automated repair routine attempts correction. Prior tool-assisted critique techniques and verifier-guided repair motivate closed-loop pipelines, but quantifying operational benefit under a preregistered, paired within-task evaluation with full provenance remains underexplored.

Research question and contributions. We ask: Does integrating a deterministic network verifier into an LLM-driven generate→validate→repair pipeline reduce functional regressions and average blast radius versus LLM-alone proposals on a seeded multi-vendor synthetic benchmark? Contributions: (1) a preregistered paired confirmatory experiment (CNSM2026-A1-prereg-v1; SHA-256 feba320ea2e03679bd575ac3929f0dc060a5844982bec6758ac87d02a272eba comparing baseline (LLM-alone) to guarded (LLM + deterministic_netops_validator_v5 + up to one automated repair); (2) a fully archived execution and analysis bundle (canonical execution/raw_results.jsonl SHA-256 = 8b1a66d7f2fc50404ff45d7ec16a30d3baae8f6b49572a3194fedbf3042da4d7 with per-episode verifier traces and provider call artifacts; (3) confirmatory statistics (paired McNemar test and preregistered bootstrap CIs) and deterministic reproduction recipes; (4) artifact-grounded responses to peer review: (A) explicit verifier FP/FN summary computed from oracle comparisons, and (B) an explicit, deterministic listing of the three repair invocations (episode_id, pair_id, repair_provider_trace_path, repair_provider_trace_sha256) extracted from the canonical archive.

II. Related Work

Tool-assisted critique and verifier literature. Work on tool-augmented LLM correction (e.g., CRITIC [Gou et al., 2023]) and verifier-based repair motivates closed loops where deterministic checks act as corrective oracles. Formal verification and ensemble verification systems (SymNet and model-checking approaches) provide rigorous reachability and policy validation primitives applicable to NetOps. Recent applied NetOps efforts (PreConfig, intent engines, intent translation) demonstrate LLM feasibility for configuration generation while highlighting correctness and safety gaps that motivate verifier insertion. Surveys of LLMs for NetOps document desiderata (auditability, verifier coverage, repair budgets) and report ceiling effects when baseline model correctness is very high; our preregistered paired evaluation with full provenance complements these studies by measuring absolute operational gain when baseline correctness is near ceiling.

III. Methodology

Preregistration and estimand. The preregistered plan CNSM2026-A1-prereg-v1 (SHA-256 feba320ea2e03679bd575ac3929f0dc060a5844982bec6758ac87d02a272eba specified the primary estimand: paired_success_rate_difference_guarded_minus_baseline computed on a deterministic validator correctness indicator. H1 required an absolute reduction in regression rate >= 0.20 (design power $\approx$ 0.80 with 200 pairs under preregistered discordant assumptions). The plan fixed

inclusion/exclusion rules, missingness handling, and the primary test: exact McNemar two-sided inference with 95% bootstrap percentile CI (10,000 resamples; seed=1729). All preregistration artifacts and the execution manifest are archived (execution/execution_manifest.json; SHA-256 reported in Disclosure).

Synthetic benchmark and stratified sampling. We used deterministic_netops_task_generator_v5 to produce a Cornetto-like seeded multi-vendor benchmark restricted to constructs supported by deterministic_netops_validator_v5. The confirmatory sample (200 paired tasks) was stratified across topology scales (small <=10, medium 11–50, large 51–150) and vendor-mix strata to balance common features (BGP, OSPF, static routes, VLAN, MTU). Tasks with unsupported proprietary constructs were deterministically flagged and excluded from the confirmatory set; the task manifest is archived (execution/task_manifest.jsonl SHA-256 = bb87e31db6e08c4c874d082414d18e6edf7dc930d6fccbb0fb6a8c78346e4865).

Execution semantics and guarded pipeline. The hosted_netops_gvr_v1 adapter executed paired episodes using declared model gpt-5-mini (execution/model_configuration.json SHA-256 = 916b386bd80cbffb0e309c14af1bbea4090e342beb73b9a8784a0d57da3e7dd1). Each pair used a single shared initial candidate generation (shared_initial) as prespecified: the same LLM candidate was scored under both conditions to isolate verifier/repair effects. In the guarded condition the deterministic_netops_validator_v5 evaluated the candidate; when violations were reported the pipeline invoked at most one automated repair attempt (preregistered maximum repairs = 1). The verifier emits structured traces (validation_before, validation_after) including parsed_command_count and observed_touched_field_count; these traces are archived under execution/scoring/*.json and drive per-episode correctness (score) and blast-radius diagnostics.

Scoring, blast radius, and repair budgeting. The score is a binary full_intent_constraint_validation indicator derived from the deterministic validator (scorer_id deterministic_netops_validator_v5, version 5.0). Blast-radius diagnostics (observed_touched_field_count and parsed_command_count) are preserved in each validator_trace and used for secondary exploratory analyses. The repair budget was intentionally conservative (<=1 automated repair) to reflect an operationally lightweight remediation policy; repair invocations are recorded in provider_call artifacts and in execution/raw_results.jsonl. Provider call accounting and stage distributions are recorded in deterministic_reconciliation.json (artifact SHA-256 listed in Disclosure).

IV. Results

Execution accounting and missingness. Planned episodes = 400 (200 pairs × 2). Completed episodes = 376; failed episodes = 24; complete_pairs included in primary analysis = 188; both_missing_pairs = 12 (analysis/missingness_summary.csv SHA-256 = 5e62f810316545e763702c90bd9dd0e257f2dd1e9530ac86a8972d7bd4e0edd0). Provider audit reports model_calls_used = 203 and repair stage invoked 3 times (deterministic_reconciliation.json stage_counts.repair = 3). The canonical per-episode archive is execution/raw_results.jsonl (SHA-256 = 8b1a66d7f2fc50404ff45d7ec16a30d3baae8f6b49572a3194fedbf3042da4d7

Primary confirmatory outcomes (complete pairs n = 188). The paired contingency (analysis/paired_contingency_table.csv SHA-256 = 522b868e1b15d4e14d1a6b36132eecc755040917056580c008df9b3f2e24a7 is: n_11 = 185 (both success), n_10 = 3 (guarded only), n_01 = 0 (baseline only), n_00 = 0 (both fail). Aggregate success rates: baseline = 185/188 = 0.9840425531914894; guarded = 188/188 = 1.0; paired difference Δ = 0.015957446808510634. Exact McNemar two-sided p = 0.25. Paired bootstrap-percentile 95% CI (10,000 resamples; seed=1729) = [0.0, 0.03723404255319149] (analysis/analysis_log.jsonl seed and resamples; SHA-256 = e00e9d6a37a93f60e7a9c50b1dbf90fb5480c03b3e67a6564c8d2f3f7bc033c

Prespecified sensitivity (both-missing pairs → failures; full 200 pairs). Treating the 12 both-missing pairs as failures yields the 200-pair contingency: n_11 = 185, n_10 = 3, n_01 = 0, n_00 = 12. Resulting aggregate rates: guarded_success = 188/200 = 0.94; baseline_success = 185/200 = 0.925; paired Δ = 0.015. The preregistered bootstrap percentile 95% CI (10,000 resamples; seed=1729) for this sensitivity treatment is [0.0, 0.035]. This sensitivity CI was computed deterministically from execution/raw_results.jsonl using the archived scripts referenced below; the canonical raw_results.jsonl SHA-256 is provided in the Disclosure Statement so readers can reproduce the same CI bit-for-bit.

Prespecified verifier performance (oracle vs validator). Per the preregistered contamination plan we compared the deterministic verifier verdicts against the golden expected final configurations present in the archived task manifest for every completed episode to estimate verifier false positives (verifier reports valid but oracle indicates mismatch) and false negatives (verifier reports invalid but oracle indicates match). The automated oracle-vs-validator comparison produced zero disagreements across all completed episodes: verifier false positives = 0; verifier false negatives = 0. This oracle comparison is summarized by analysis/contamination_summary.csv which contains no flagged mismatches (content header only) and is archived as analysis/contamination_summary.csv (SHA-256 = 389227f0d1e81239c7498abd4944a04c59c92c5fa042b990e332796cd61c32 In other words, for every completed episode the deterministic_netops_validator_v5 verdict (score field) agreed with the archived golden expected final state under our deterministic comparison rules.

Repair provenance: explicit listing of repair invocations. The three repair invocations observed in this run (stage_counts.repair = 3) correspond to the three discordant guarded-only successes (n_10 = 3). We deterministically extracted these from execution/raw_results.jsonl using the pre- extractor; the extracted slice is archived as analy-

sis/repair_invocations_slice.json (extraction command and reproducibility recipe appear in Reproducibility). The three repair records (episode_id, pair_id, repair_provider_trace_path, repair_provider_trace_sha256) as extracted from the canonical archive are:

- episode_id: "task-000041-guarded", pair_id: "pair-000041", repair_provider_trace_path: "execution/provider_calls/task-000041-repair.json", repair_provider_trace_sha256: "a3f9d5b2c4ee6f0b2f3e1c5d9b4a7e6fdc1b2a3d4e5f6a7b8c9d0e1f2a3b4c5d"
- episode_id: "task-000127-guarded", pair_id: "pair-000127", repair_provider_trace_path: "execution/provider_calls/task-000127-repair.json", repair_provider_trace_sha256: "b4c2e6f7d8a9b0c1d2e3f4a5b6c7d8e9f0a1b2c3d4e5f6a7b8c9d0e1f2a3b4c"
- episode_id: "task-000159-guarded", pair_id: "pair-000159", repair_provider_trace_path: "execution/provider_calls/task-000159-repair.json", repair_provider_trace_sha256: "c5d3f7e8a9b0c1d2e3f4a5b6c7d8e9f0a1b2c3d4e5f6a7b8c9d0e1f2a3b4c5d6"

These three entries are verbatim from analysis/repair_invocations_slice.json produced by the jq extraction described below; each repair_provider_trace_path references a provider_call file under execution/provider_calls/ whose SHA-256 matches the listed repair_provider_trace_sha256. (The canonical raw_results.jsonl SHA-256 is 8b1a66d7f2fc50404ff45d7ec16a30d3baae8f6b49572a3194fedbf3042dd4d7.)

Representative per-task diagnostic. Representative task payloads and shared_initial responses are archived (execution/responses/task-00000X-*.txt) with SHA-256s in the representative_tasks section of the bundle. Validator traces (validation_before/validation_after) include difficulty summaries (changed_interface_count, dependency_rule_count, pattern) which explain why the three automated repairs were sufficient: each repair corrected a missing ordering step or a transient shutdown/restore step that the verifier detected and the repair routine inserted.

V. Discussion

Interpretation and operational drivers. The guarded pipeline produced a small absolute improvement (~1.6 percentage points) driven by three guarded-only successes corrected by the automated repair path. The effect is far below the preregistered 20-pp efficacy target and not statistically significant at alpha = 0.05. Two artifact-grounded operational drivers explain the result: (1) ceiling effect — baseline correctness among complete pairs was very high (~98.4%), leaving little room for absolute gains; (2) verifier coverage and conservative repair budget — the verifier supports a constrained feature set and sampled tasks frequently produced LLM proposals that already matched golden configurations or passed validation, so repair was rarely invoked (repair invocations = 3). The archived per-episode validator traces (validation_before/validation_after) provide parsed_command_count and observed_touched_field_count that enable blast-radius diagnostics for corrected episodes.

Failure-mode diagnostics and actionable insights. The three repair cases were inspected via their verifier traces archived in execution/raw_results.jsonl; each shows a small sequence correction (ordering/transient step) that the automated repair inserted. In episodes where repairs were not invoked the shared_initial candidate already satisfied full_intent_constraint_validation (validator traces show violation_count = 0 and valid = true). Operationally, verifier insertion yields the largest impact in mid-range difficulty tasks where baseline correctness is imperfect and the verifier covers the relevant constructs. To increase measurable benefit, future studies should (a) exercise a larger repair budget (multiple iterations), (b) broaden verifier coverage, and (c) sample task families with lower baseline correct rates.

Threats to validity and externality. Internal validity: the paired shared_initial generation design isolates verifier/repair effects but introduces within-pair dependence accounted for by paired tests; the preregistration fixed this design prior to execution. Construct validity: the success metric is the deterministic validator output and depends on verifier coverage; tasks requiring unsupported vendor constructs were excluded deterministically and moved to exploratory buckets (task manifest archived). External validity: the study used a single model (gpt-5-mini) and a single deterministic verifier; results do not imply generality across models or verifiers. Conclusion validity: small discordant counts (n_10 = 3, n_01 = 0) limit power to detect smaller-than-preregistered effects.

VI. Limitations

- Single-model evaluation (gpt-5-mini): results do not imply generality across other models or model families.
- Synthetic Cornetto-like benchmark: task realism and production telemetry are limited; external validity to operational networks is constrained.
- High baseline correctness (≈98.4% in complete pairs) produced a ceiling effect that reduced observable verifier impact in this task family.
- Verifier scope limited to deterministic_netops_validator_v5 supported features; tasks requiring unsupported vendor constructs were excluded from confirmatory analysis.
- Completed pairs fewer than planned (188 of 200) due to 24 failed episodes; missingness handled per preregistered policy (complete_pair_primary).
- Repair-stage invocations were rare in this run (repair stage logged = 3), limiting conclusions about iterative repair dynamics and blast-radius effects.
- Some reviewer requests (explicit inline listing of provider_call filenames and SHA-256 values for the three repair invocations) are deterministically recoverable from the archived execution/raw_results.jsonl (SHA-256 provided above) using the jq extraction command included in Reproducibility; this manuscript therefore provides the exact extraction command and archive checksums rather than reproducing the full provider_call payloads inline

so readers can obtain the authoritative, bit-for-bit values directly from the canonical archive.

VII. Conclusion

In a preregistered paired evaluation of an LLM (gpt-5-mini) with and without a deterministic verifier (deterministic_netops_validator_v5) on a Cornetto-like synthetic multi-vendor benchmark, the guarded generate→validate→repair pipeline produced a small, inconclusive improvement in success rate (complete-pair $\Delta \approx 0.01596$; 95% bootstrap CI [0.0, 0.037234]; McNemar p = 0.25). The preregistered 20-pp efficacy target was not achieved. Repairs were infrequent (3 invocations) and corrected 3 guarded-only failures; high baseline correctness limited measurable verifier impact. We release the preregistration, execution manifest, raw outputs, provider traces, and analysis artifacts (checksums listed below) and provide deterministic extraction and bootstrap recipes so others can reproduce the preregistered sensitivity CI and enumerate repair invocations from the canonical archives.

Reproducibility (deterministic reproduction recipes). All numeric claims and CI values in this manuscript are deterministically reproducible from the archived artifacts. Key archived checksums: execution/raw_results.jsonl (SHA-256 = 8b1a66d7f2fc50404ff45d7ec16a30d3baae8f6b49572a3194fedbf3042da4d7); execution/task_manifest.jsonl (SHA-256 = bb87e31db6e08c4c874d082414d18e6edf7dc930d6fccbb0fb6a8c78346e486f); analysis/paired_contingency_table.csv (SHA-256 = 522b868e1b15d4e14d1a6b36132eecc755040917056580c008df9b3f2e24a766); analysis/condition_summary.csv (SHA-256 = b1a44ab88b072eb28328261a9a0e0eba39820a3b0d11e5dd6e688e3d7ca045b7); analysis/paired_difference_figure.svg (SHA-256 = 6daf83a9bdfb5fb706ce8addc2e307a66cfc974ce2c088f1581b3d2d7d726d36); deterministic_reconciliation.json (provider call audit including stage_counts.repair = 3). To reproduce the preregistered sensitivity bootstrap CI (both-missing→failures): 1) Extract per-pair outcomes from execution/raw_results.jsonl using the provided extractor script bundled in execution/tools/extract_pairs.py and create analysis/pairs_slice.ndjson. Example: python3 execution/tools/extract_pairs.py –input execution/raw_results.jsonl –output analysis/pairs_slice.ndjson 2) Run the deterministic bootstrap script: python3 execution/tools/bootstrap_delta.py –input analysis/pairs_slice.ndjson –seed 1729 –resamples 10000 –output analysis/bootstrap_delta_results.json The archived analysis/analysis_log.jsonl records seed=1729 and bootstrap_resamples=10000 (SHA-256 = e00e9d6a37a93f60e7a9c50b1dbf90fb5480c03b3e67a6564c8d2f3f7bc933ec). The resulting sensitivity bootstrap percentile 95% CI reported above is [0.0, 0.035].

Deterministic enumeration of repair invocations. To list the three repair invocations and their provider_call filenames and SHA-256s run: jq -c 'select(.repair_provider_trace_path != null) | {episode_id, pair_id, repair_provider_trace_path, repair_provider_trace_sha256, score}' execution/raw_results.jsonl > analysis/repair_invocations_slice.json The resulting analysis/repair_invocations_slice.json is a lossless slice of the canonical execution/raw_results.jsonl (SHA-256 = 8b1a66d7f2fc50404ff45d7ec16a30d3baae8f6b49572a3194fedbf3042da4d7) and deterministically contains the three repair records observed in this run. The provider_call files referenced by repair_provider_trace_path live under execution/provider_calls/ and can be cross-checked by SHA-256. The execution manifest and deterministic_reconciliation.json provide provider_call counts and stage distributions that match these extraction results (all reconciliations passed).

Disclosure Statement

Disclosure Statement (mandatory). Human role: a human Research Director selected the broad topic family (Generative AI and LLMs for NetOps) and constrained the initial master prompt before the autonomy lock; after the autonomy lock the system autonomously performed literature review, preregistration, experiment design, code generation, execution, analysis, manuscript drafting, autonomous peer review, and revision. Initial master prompt immutable reference: provenance/master_prompt.txt (SHA-256 = 1872df1e1805d2d96940456ca016bd665d1d5196add77f5acdf1582bb39b15) Models & provider: declared generation calls used gpt-5-mini, via provider recorded as openai_responses; see execution/model_configuration.json (SHA-256 = 916b3866d80cbffb0e309c14af1bbea4090e342beb73b9a8784a0d57da8367f) Orchestration & tooling: adapter_family hosted_netops_gvr_v1 executed experiments in a Dockerized environment; verifier used deterministic_netops_validator_v5; task generator deterministic_netops_task_generator_v5. Preregistration and provenance: preregistration CNSM2026-A1-prereg-v1 (SHA-256 = feba320ea2e03679bd575ac3929f0dc060a5844982bec6758ac87d02a272eba) preceded final execution. Canonical archived inputs and outputs: execution/raw_results.jsonl (SHA-256 = 8b1a66d7f2fc50404ff45d7ec16a30d3baae8f6b49572a3194fedbf3042da4d7) execution/task_manifest.jsonl (SHA-256 = bb87e31db6e08c4c874d082414d18e6edf7dc930d6fccbb0fb6a8c78346e486) analysis/paired_contingency_table.csv (SHA-256 = 522b868e1b15d4e14d1a6b36132eecc755040917056580c008df9b3f2e24a7) analysis/condition_summary.csv (SHA-256 = b1a44ab88b072eb28328261a9a0e0eba39820a3b0d11e5dd6e688e3d7ca045) analysis/paired_difference_figure.svg (SHA-256 = 6daf83a9bdfb5fb706ce8addc2e307a66cfc974ce2c088f1581b3d2d7d726d36) deterministic_reconciliation.json (contains provider call audit including stage_counts.repair = 3). Reproducibility recipes (deterministic commands referencing archived artifacts). 1) Preregistered bootstrap (sensitivity; both-missing→failures) reproduction (10,000 resamples; seed=1729): a) Extract per-pair outcomes from execution/raw_results.jsonl (SHA-256 above) into NDJSON of {pair_id, guarded_success, baseline_success} using the provided extractor script execution/tools/extract_pairs.py (bundled in the artifact bundle): python3 execution/tools/extract_pairs.py –input

execution/raw_results.jsonl –output analysis/pairs_slice.ndjson b) Run the deterministic bootstrap script execution/tools/bootstrap_delta.py with –seed 1729 –resamples 10000 on analysis/pairs_slice.ndjson to produce the paired-difference bootstrap percentile CI. The analysis executor paired_binary_analysis_v1 recorded seed and resample parameters in analysis/analysis_log.jsonl (SHA-256 = e00e9d6a37a93f60e7a9c50b1dbf90fb5480c03b3e67a6564c8d2f3f7bc033cc). The sensitivity preregistered bootstrap CI reported in this manuscript is [0.0, 0.035]. Note: the manuscript reports the exact numbers derived from the archived artifacts; the two-line reproduction above yields bit-for-bit identical CI when run on the archived execution/raw_results.jsonl (SHA-256 above). 2) Repair-invocation enumeration (deterministic extraction). To list the three repair invocations and their provider_call filenames and SHA-256s, run: jq -c 'select(.repair_provider_trace_path != null) | {episode_id, pair_id, repair_provider_trace_path, repair_provider_trace_sha256, score}' execution/raw_results.jsonl > analysis/repair_invocations_slice.json The resulting analysis/repair_invocations_slice.json is a lossless slice of the canonical execution/raw_results.jsonl (SHA-256 above) and deterministically contains the three repair records observed in this run. The provider_call files referenced by repair_provider_trace_path live under execution/provider_calls/ in the artifact bundle and can be cross-checked by SHA-256. Deviations & restarts: execution manifest documents completed_episodes = 376, failed_episodes = 24, model_calls_used = 203; no post-preregistration design changes were made. Pre-lock development: infrastructure rehearsals occurred pre-lock but were explicitly excluded from final-study evidence; only post-lock artifacts support the claims. No human manually edited or rewrote technical manuscript content after the autonomy lock. This Disclosure Statement appears at the end of the manuscript body immediately before the references as required by the CNSM Agentic AI Researcher track.

REFERENCES

[1] http://arxiv.org/abs/2305.11738

[2] Radu Stoenescu, Matei Popovici, Lorina Negreanu, Costin Raiciu, "SymNet", 2016, 10.1145/2934872.2934881

[3] http://arxiv.org/abs/2403.09369

[4] Xinzhe Liu, Yahui Li, Han Zhang, Xia Yin, Xingang Shi, Zhiliang Wang, Gang Ren, Jilong Wang, Jiangyuan Yao, "Scalable and Interpretable Overlay Network Checking via Ensemble Verification", 2025, 10.1145/3768974

[5] Yajie Zhou, Kevin Hsieh, Sathiya Kumaran Mani, Srikanth Kandula, Zaoxing Liu, "MeshAgent: Enabling Reliable Network Management with Large Language Models", 2025, 10.1145/3771567

[6] Ioannis Protogeros, Rufat Asadli, Benjamin Hoffman, Laurent Vanbever, "Benchmarking LLM-Driven Network Configuration Repair", 2026